EUUI -- 全局/单人面板

创建第一个面板

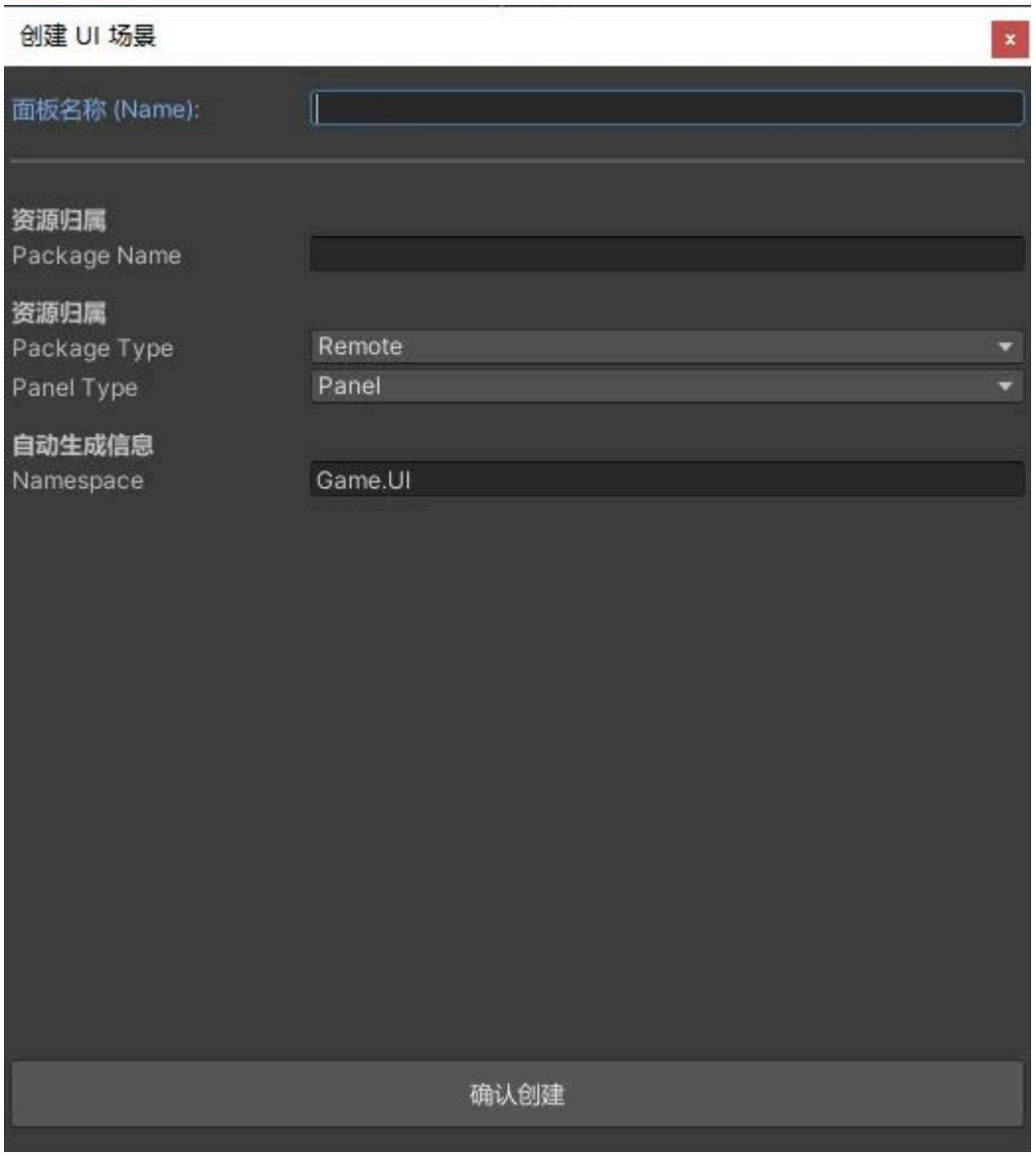
有两种方式可以唤出创建 UI 场景的的面板：

方法一：使用快捷键按键盘快捷键 `Alt + U` 即可直接唤出面板。

方法二：通过功能编排

1. 在功能编排中，将“创建UI场景”编入功能区域。
2. 编译完成后，在 Scene 场景中按下 `Space` 键唤出热盒（Hotbox）。
3. 点击热盒中的“创建UI场景”按钮。

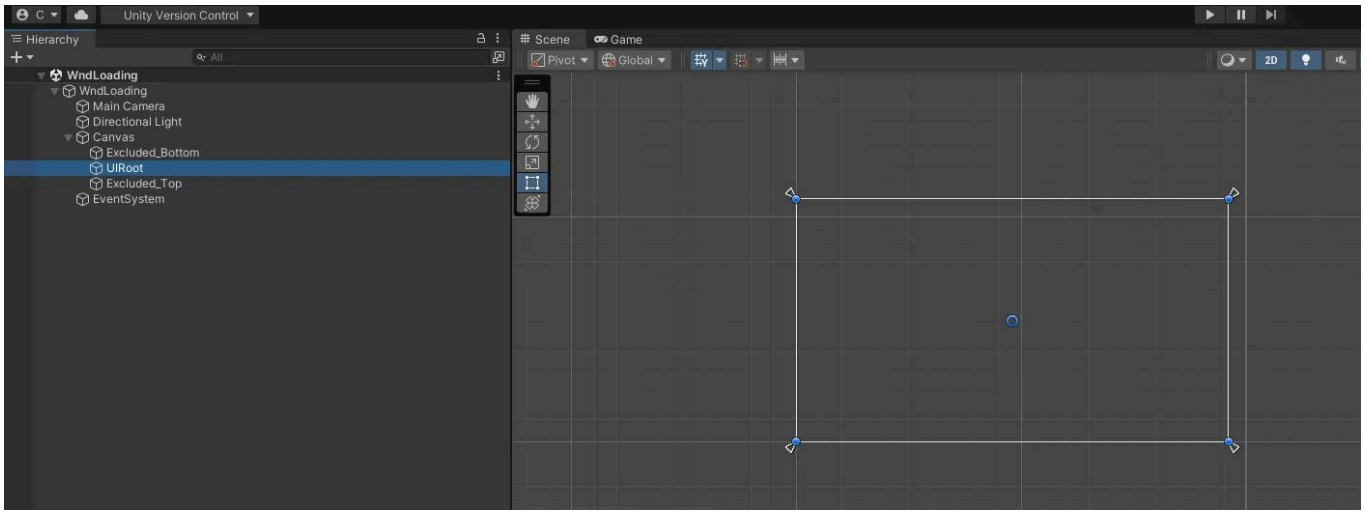
填写创建面板参数



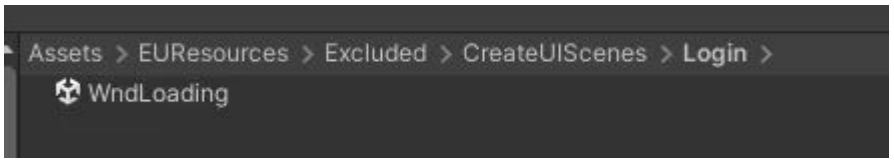
唤出面板后，请准确填写以下配置项：

- **面板名称**：输入要创建的面板名称，例如 `WndLoading`、`WndStart`。
- **资源归属**：指定此面板所属的模块（用于资源整合）。例如，若资源属于登录模块，则在 `PackageName` 字段填入 `Login`。最终文件会生成在 `prefix/packageName` 目录下。
- **PackageType**：通常选择默认（Remote）即可。Remote 包在分包处理时，资源不会随主包体打包。除非你确定该资源必须随主包发布，才应选择 Builtin。
- **PanelType**：选择面板类型，即导出代码的基类。
  - 普通面板选择 `Panel`。
  - 弹框选择 `Popup`。
  - 包含上下栏的界面选择 `Bar`。
- **Namespace**：导出代码的命名空间。默认会读取配置文件中的数据，保持默认即可。

完成填写后，点击创建，系统会自动生成相关资源文件。

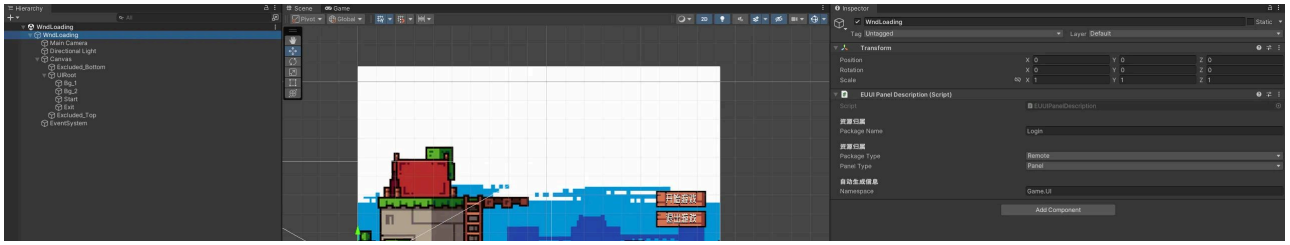


请注意：创建的资源存储文件夹仅供生成器内部使用，其中的资源不会直接导出，请不要在此文件夹内直接使用或修改资源。



创建的资源存储，这个文件夹的资源不会导出，不要直接使用

可以在根目录下修改创建时填入的信息，这些信息是导出使用的，并不会影响UIScene

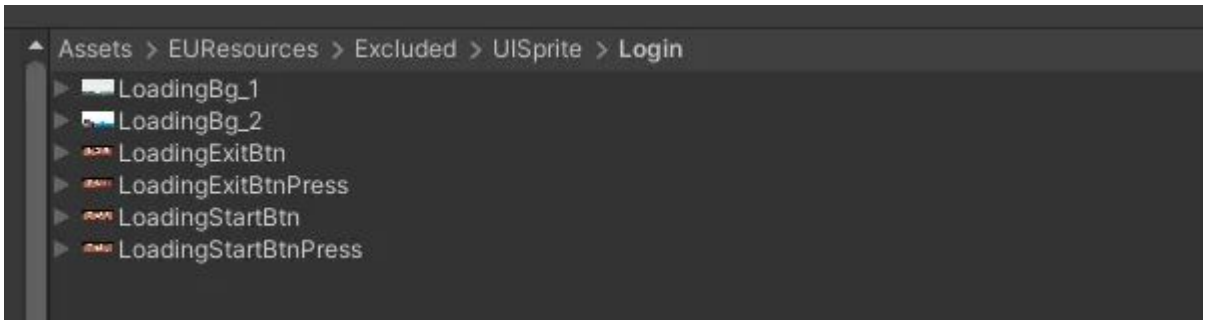


导入资源

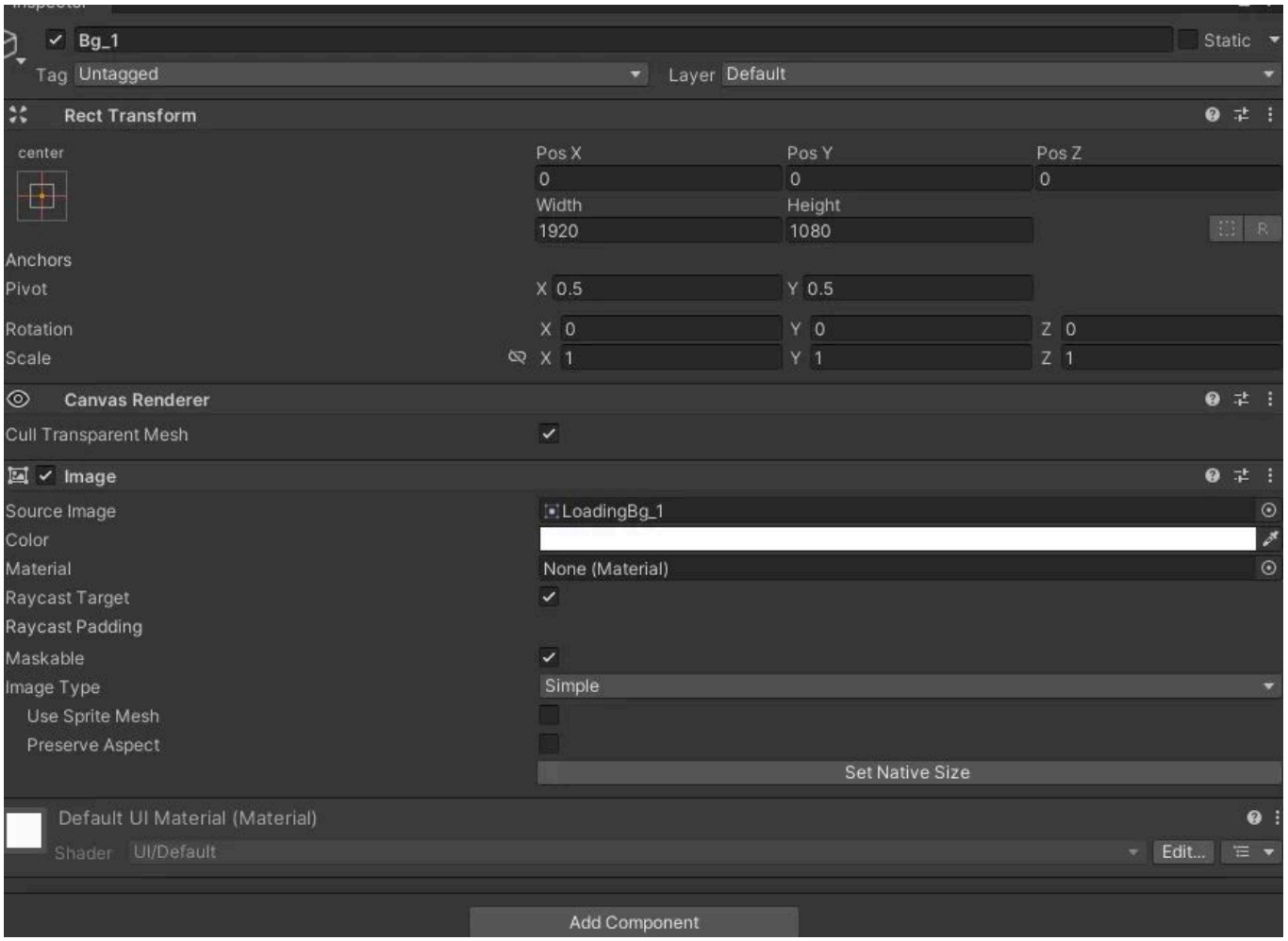
1. 资源归属判定与存放

导入资源前，首先需明确其归属：

- **公共资源**：若该资源被多个跨模块的UI面板共同使用，应存放于`Common`目录下。
- **专属资源**：若资源仅服务于特定模块的UI面板，则需与该模块保持一致。
  - **操作步骤**：在资源文件夹（推荐使用`Excluded/UISprite/`等非导出资源目录）中，创建对应模块的子文件夹（如`Login`同使用面板的`PackageName`），再将所需资源移入其中。



资源成功放入指定文件夹后，即可直接在控件上使用。



2. 打包与生成图集

为优化性能，需要将导入的图片资源打包成图集。**打开图集生成面板前，请先选中包含待打包图片的文件夹**。

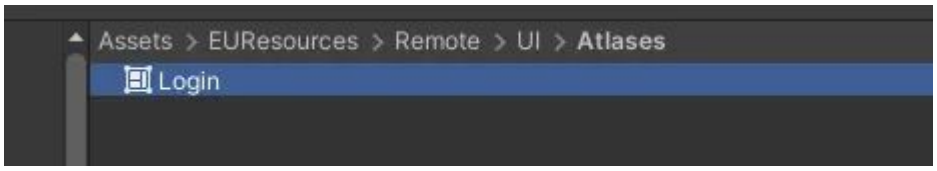
方法一：快捷键打开按下快捷键 `Ctrl + Alt + G`。

方法二：通过功能编排打开

1. 在“功能编排”中，将“生成图集”功能编入功能区域。
2. 完成编译后，在任一`Scene`场景中按下 `Space` 键唤出热盒（Hotbox）。
3. 点击热盒中的“生成图集”按钮。

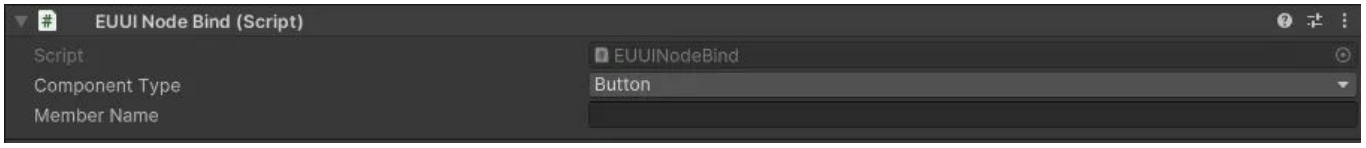
随后会弹出选项窗口，请根据该UI面板资源的用途（放入远程包或首包）进行选择。

点击“生成资源”按钮，完成图集创建。



绑定导出Prefab

选中场景中的目标控件，按下快捷键 `Ctrl+Alt+B`，即可为其绑定脚本。



在绑定面板中，需要配置以下关键参数：

- **Component Type**：该控件在代码中对应的组件类型。预设类型（如`OSAList`）会直接显示。若无对应选项，请选择`RectTransform`，后续在代码中手动转换。
- **MemberName**：控件在代码中使用的引用名称。此字段选填，若不填，将默认使用控件在 *Hierarchy* 中的命名。

如何调用自动绑定导出

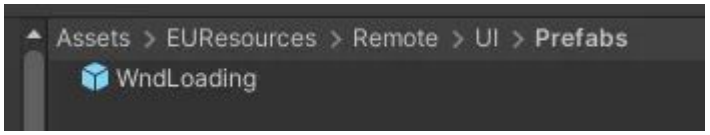
1. 在“功能编排”中，将“自动绑定导出”功能编入功能区域。
1. 完成编译后，在任何 `Scene` 场景中按下 `Space` 键，呼出热盒（Hotbox）。
2. 在热盒中点击“自动绑定导出”按钮。

导出生成结果

执行导出后，将生成以下三类核心文件：

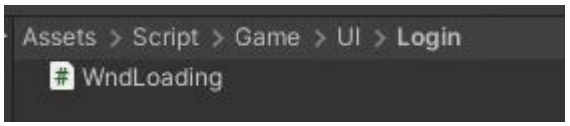
1. Prefabs 预制体

生成绑定好的预制体资源。



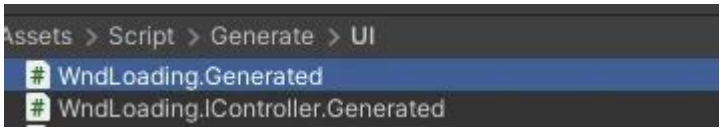
2. 业务使用代码

生成可直接用于业务逻辑驱动的接口代码（如MVC模式中的View层）。



3. UI元件生成代码

生成Assembly-CSharp内，用于实例化和绑定UI元件的基础生成代码。



此结构与MVC（Model-View-Controller）模式中的接口继承逻辑相适配。

如何调整控件绑定

对已导出的Prefab，根据修改内容的不同，采取以下操作：

- **仅修改控件位置或属性**：直接选中并编辑已导出的Prefab即可，业务逻辑代码不会被覆盖。
- **新增或移除绑定**：必须再次运行“自动绑定导出”功能。该操作会重新生成绑定代码和Prefab，但不会覆盖您已编写的业务使用代码。

创建列表项

OSA编排编辑器提供了快速创建列表项（Item）的完整工具链，主要功能包括：

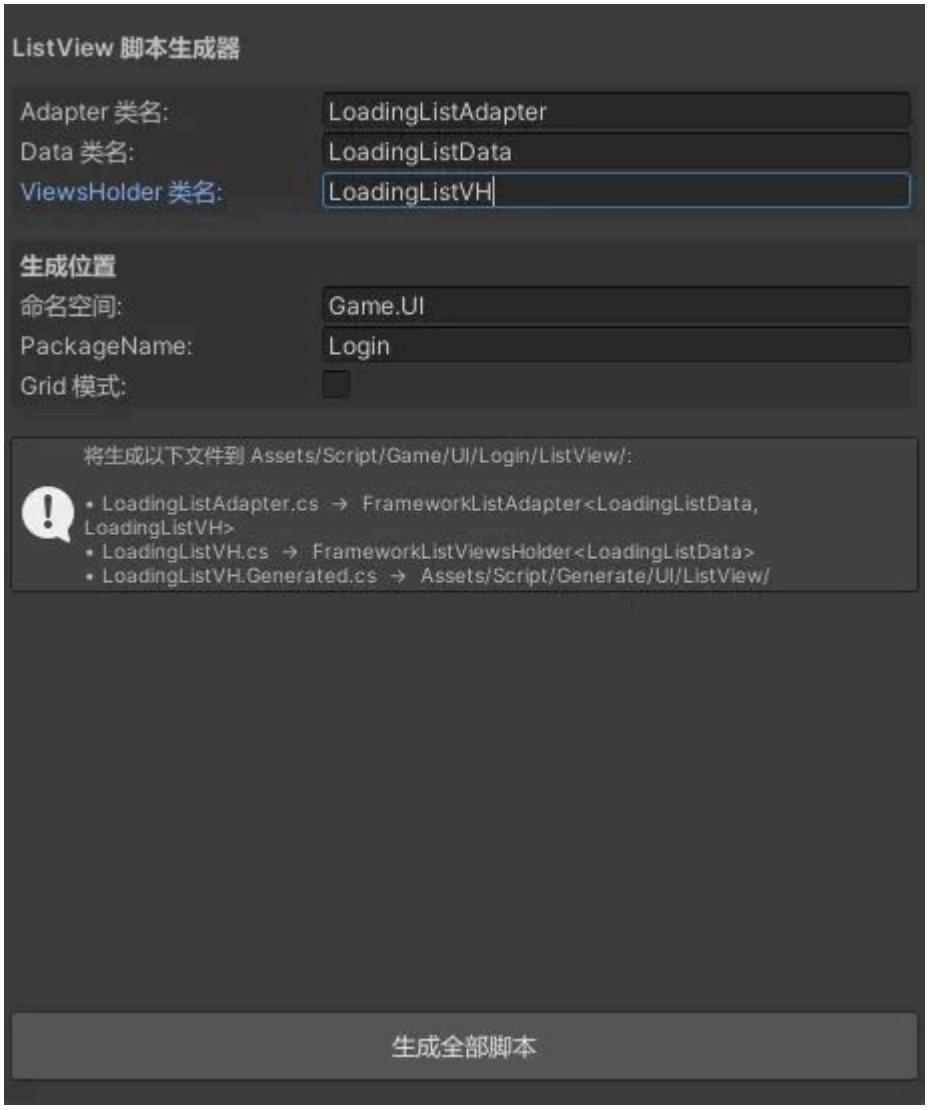
- **生成Adapter**：列表数据管理与适配的核心类。
- **生成ViewsHolder (VH)**：列表项视图的持有与逻辑处理类。
- **清理废弃VH**：移除已不再使用的视图持有器。

1. 创建 Adapter

1.1 步骤说明

1. 在OSA编排编辑器中，定位到“创建列表项”功能入口。
2. 点击“生成Adapter”。

在生成的配置界面中（通常为单独对话框），需填写以下关键信息：



1.2 命名规范与选项

命名约定：

- **Adapter**: `[列表名]Adapter` (例: `LoadingListAdapter`)
- **Data**: `[列表名]Data` (例: `LoadingListData`)
- **ViewsHolder (VH)**: `[列表名]VH` (例: `LoadingListVH`)

包名 (PackageName): 列表通常作为UI组件的一部分，其包名应遵循所属场景或业务模块的命名规则。

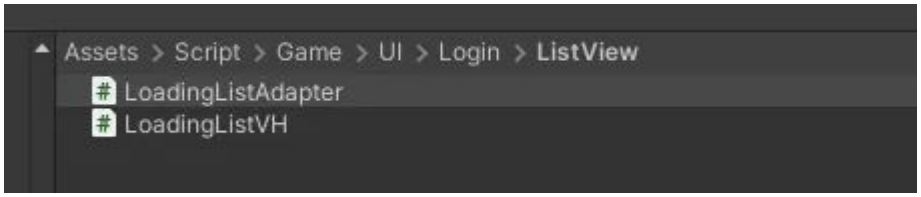
列表类型:

- 若创建的是**网格列表**（一行包含多个元素），请勾选 **Grid** 选项。

2. 创建列表项 (Item) 与容器

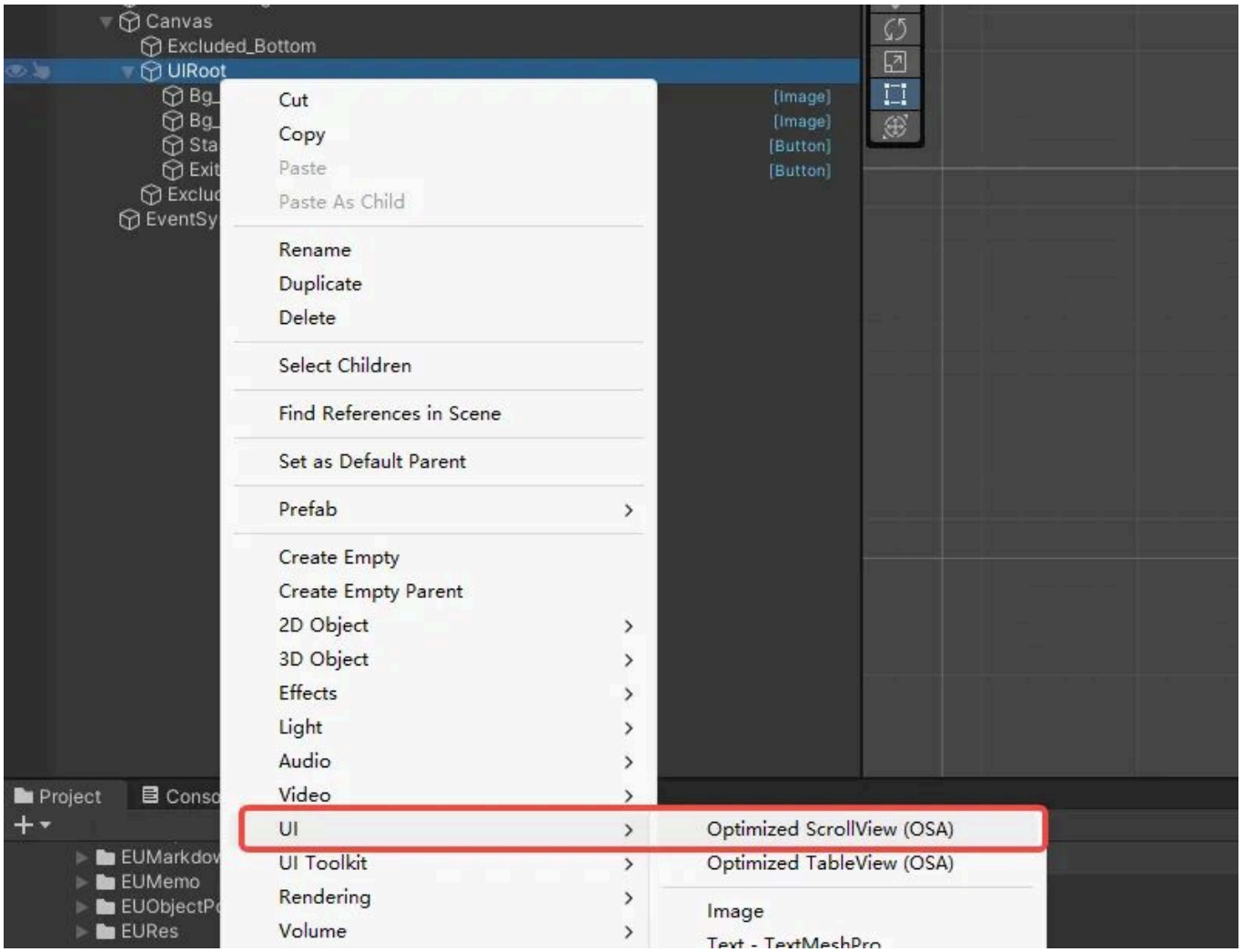
2.1 前置条件

- 确保已在项目中一个**可导出的节点**下。

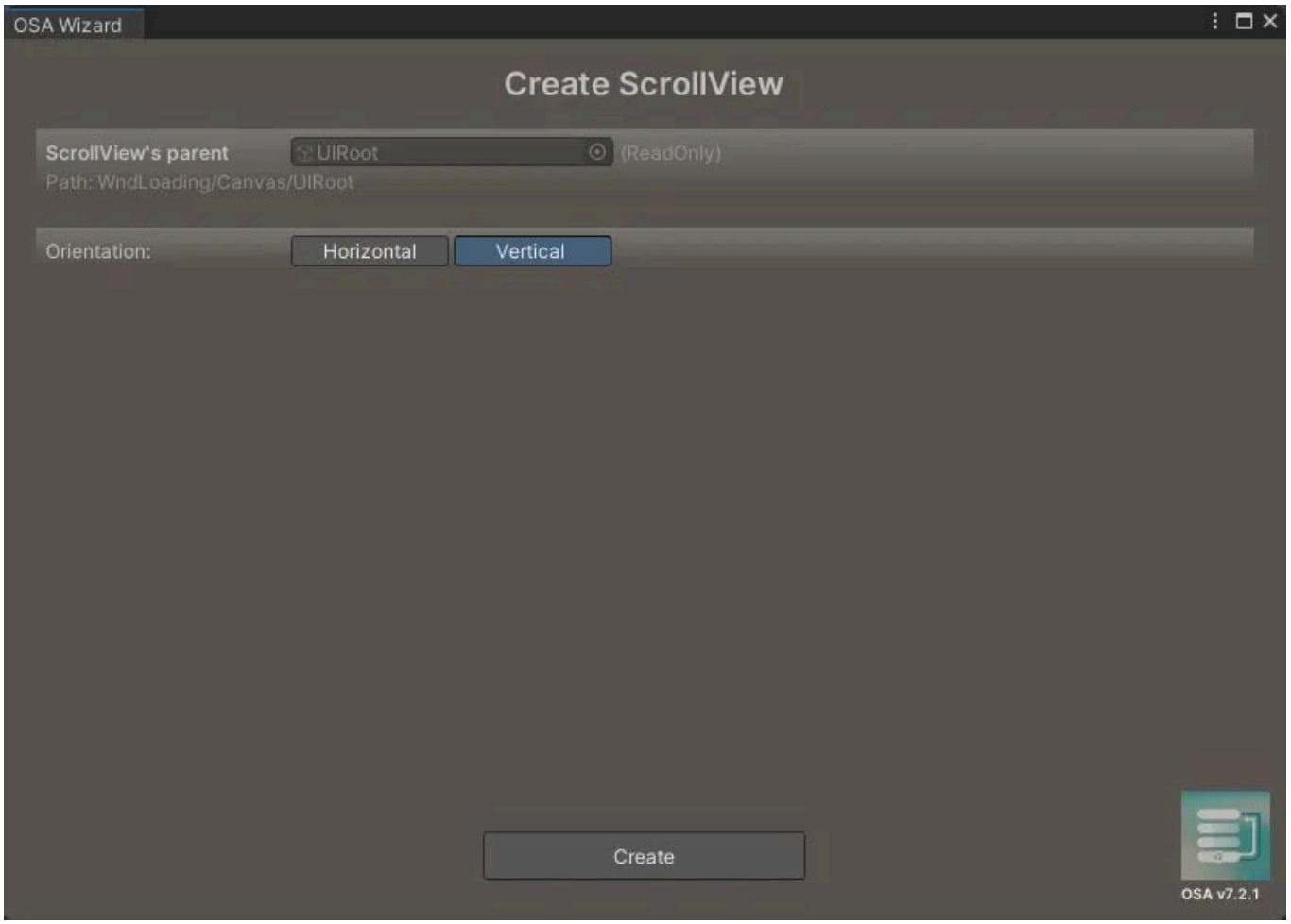


2.2 创建流程

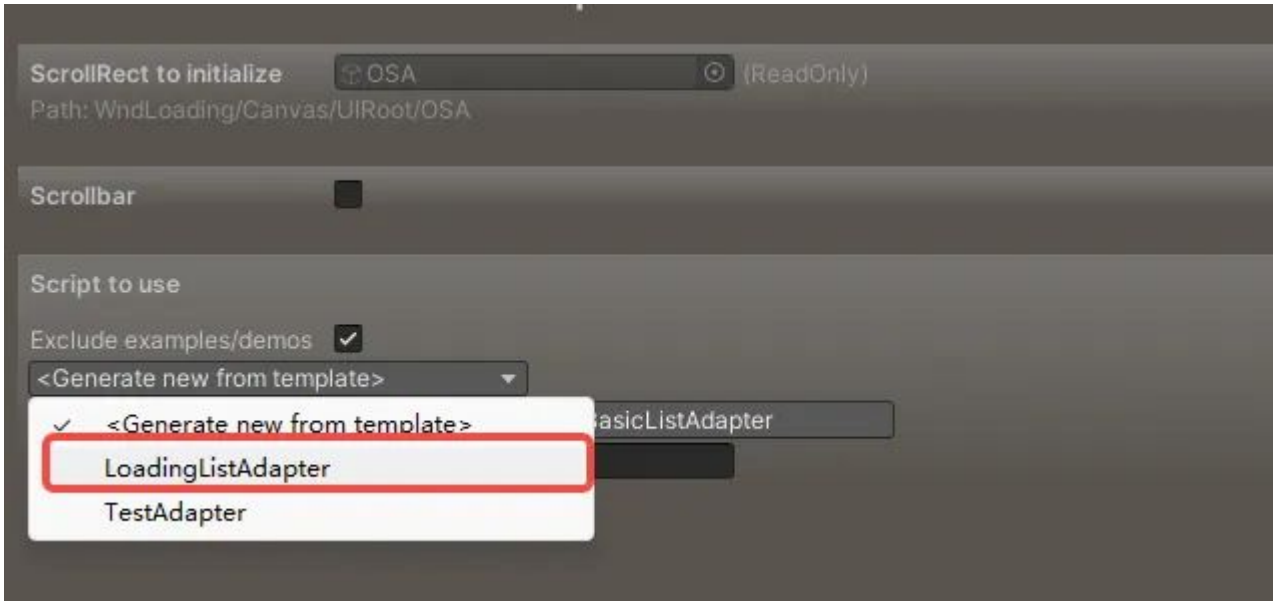
1. 在导出节点下，点击“创建List”。



2. 系统将基于OSA框架创建列表项。首先需选择列表的**滑动方向**（垂直或水平）。

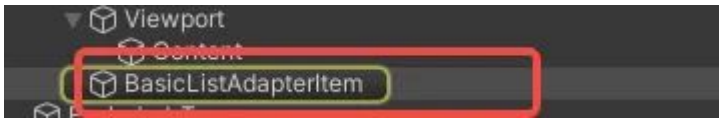


3. 在下一步中，选择**第1步已创建的Adapter**，并根据需要决定是否启用**侧边滑动栏**。

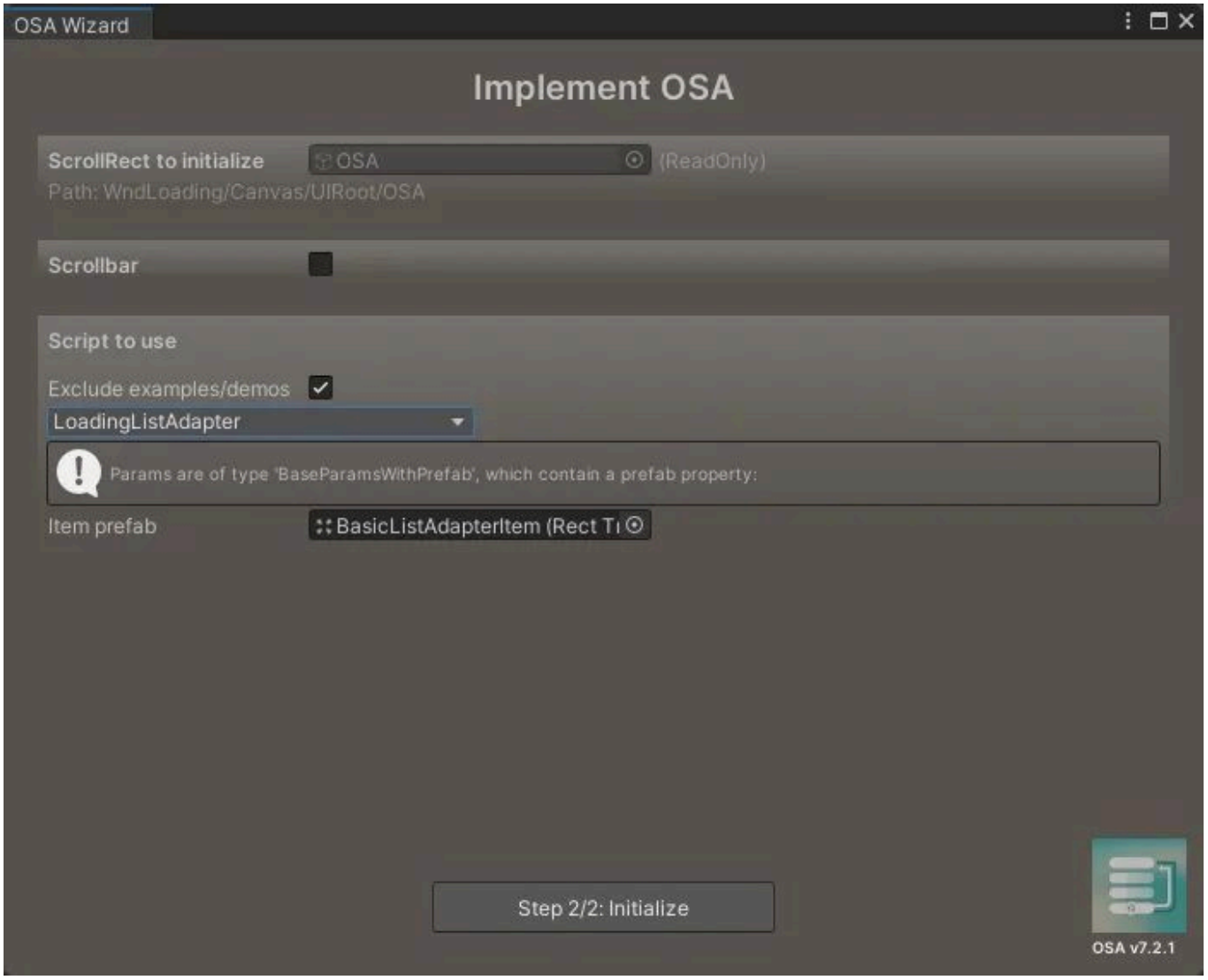


4. **Prefab处理：**
   - 如果已有可用的列表项Prefab，可直接选择。
   - 若没有，可点击 **Generate** 按钮，系统将生成一个默认Prefab供后续调整。

5. 点击完成后，系统将创建出列表项容器。

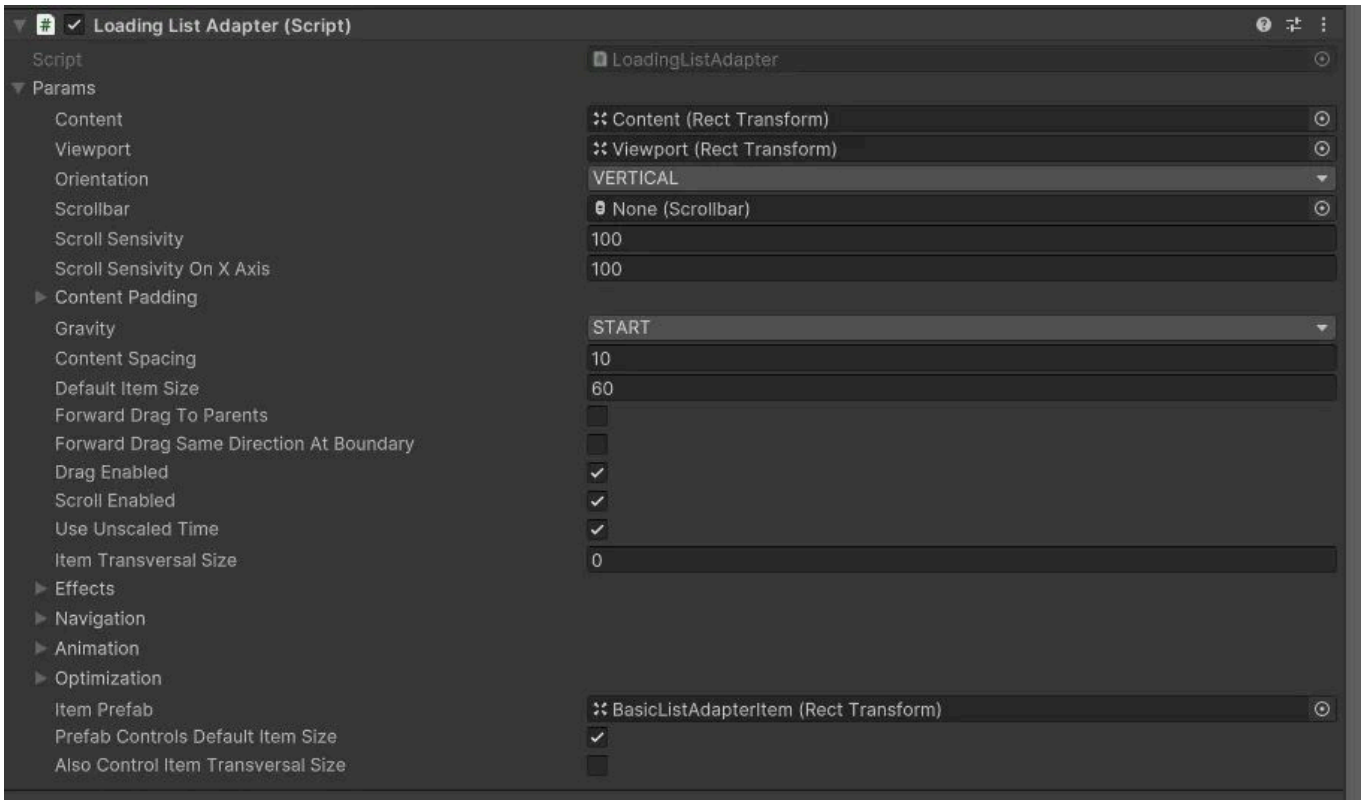


6. 确认所有配置无误后，点击 **Initialize**。



7. Initialize执行完毕，系统将根据配置生成Adapter、Data和VH的核心代码结构。

- **提示：** 在Initialize之后，上述所有配置参数仍可在生成的Adapter脚本中进行调整。



3. 定制列表项Prefab与绑定

3.1 定位与绑定

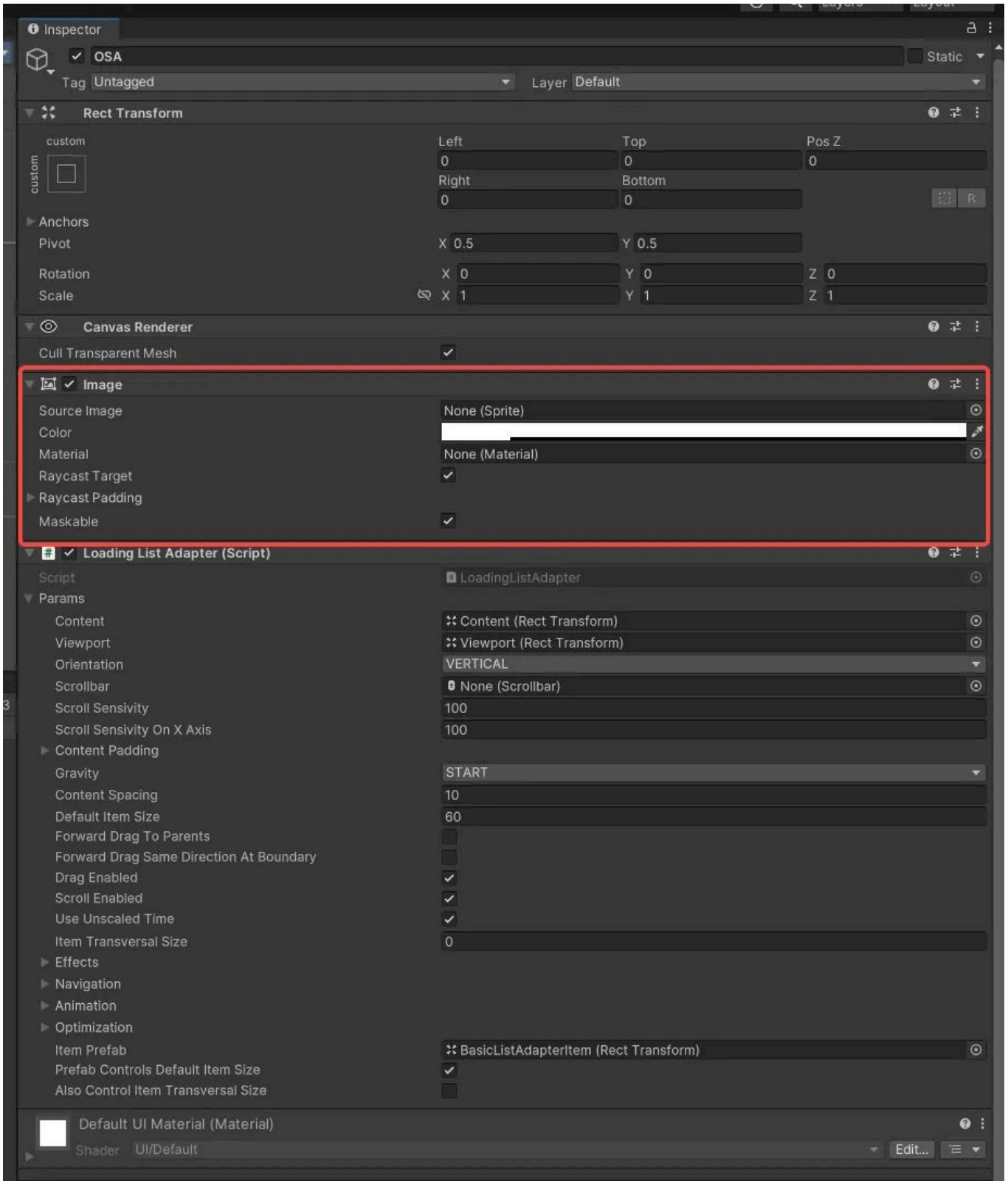
1. 在Hierarchy中选中刚刚生成的列表项（Item）。

2. 在该Item上找到由系统生成的对应VH组件。

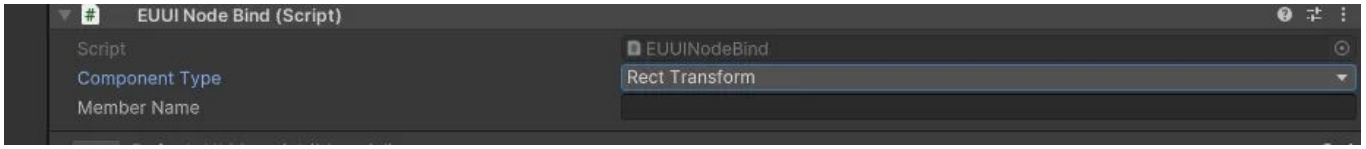
3. 点击 **生成绑定** 按钮，为Prefab内的UI控件创建数据绑定。

3.2 UI元素清理与调整

- **推荐操作：** 自动生成的列表项Prefab可能包含一个默认的Image控件作为底图。若UI设计不需要，建议**直接删除该Image控件**。



- 移除多余控件后，再次点击VH组件上的 **添加上绑定** 按钮，确保所有需要绑定的UI控件被正确关联。



3.3 重命名与最终导出

1. 根据实际业务，将生成的列表节点重命名为合适的名称。

2. 最后，执行 **一次完整的“自动绑定导出”** 操作，将Prefab的修改与代码绑定关系固化到项目中。

列表在代码中如何使用

使用代码驱动UI列表的核心流程遵循 **获取Adapter -> 准备数据 -> 绑定数据** 的通用模式。

1. 核心使用流程

在业务逻辑代码（如MonoBehaviour）中，通常按照以下三个步骤操作：

1. **获取引用**：获取场景中 `LoadingList` GameObject 上的 `LoadingListAdapter` 组件。
2. **处理数据**：构造 `LoadingListData` 对象列表作为数据源。
3. **设置数据**：调用Adapter的 `SetData` 方法，将数据传入列表系统。

代码示例

C#

```
// 1. 获取Adapter引用
var loadingList = LoadingList.GetComponent<LoadingListAdapter>();

// 2. 准备测试数据
var testData = new List<LoadingListData>();
for (int i = 0; i < 10; i++)
{
    testData.Add(new LoadingListData()
    {
        Title = "Test " + i
    });
}

// 3. 设置数据，驱动列表渲染
loadingList.SetData(testData);
```

2. Adapter 与 ViewsHolder 逻辑关系

列表系统基于 **Adapter（适配器）** 和 **ViewsHolder（视图持有器）** 两层架构，职责分离清晰。

2.1 ViewsHolder (VH) - 负责单项视图与数据绑定

ViewsHolder管理单个列表项的UI组件和生命周期。其核心任务是在 `OnAcquire` 方法中将数据对象（`LoadingListData`）绑定到具体的UI控件上。

关键点

- `OnCollectViews()`: 在生成时调用，用于查找和引用UI组件。通常只需取消注释自动生成的 `BindViews();` 即可。
- `OnAcquire(LoadingListData data, int index)`: **核心绑定方法**。列表项每次进入可视区域时调用，此处实现 `data.Title` 到 `TitleText.text` 的具体赋值逻辑。
- `OnRelease()`: 列表项移出可视区域时调用，用于可选的状态重置或资源清理。

代码示例：`LoadingListVH.cs`

C#

```
//-----------------------------------------------------------------------
// 业务逻辑部分 - 仅在初始创建时生成，请在此编写你的 ViewsHolder 逻辑
//-----------------------------------------------------------------------
using EUUI.Extension;
using frame8.Logic.Misc.Other.Extensions;
using UnityEngine;

namespace Game.UI
{
    ///
    /// LoadingListData 数据类
    ///
    public class LoadingListData
    {
        public string Title { get; set; }
    }

    ///
    /// LoadingListVH ViewsHolder
    ///
    public partial class LoadingListVH : FrameworkListViewsHolder<LoadingListData>
    {
```

```
        protected override void OnCollectViews()
        {
            // 调用自动生成的绑定方法 业务层使用时 取消注释
            BindViews();
        }

        public override void OnAcquire(LoadingListData data, int index)
        {
            // TODO: 绑定数据到 UI
            TitleText.text = data.Title;
        }

        public override void OnRelease()
        {
            // TODO: 清理资源（可选）
        }
    }
}
```

2.2 Adapter - 负责列表整体配置与交互事件

Adapter管理整个列表的行为，包括初始化参数、处理全局点击事件等。

关键点

- `Start()`: 初始化方法。可在此配置 `_Params`（如列表样式、项模板等）和事件委托。
- `_Params`: 列表的基础参数对象，允许运行时动态修改列表行为。
- `OnItemClick`: 事件委托。为其赋值即可为所有列表项统一添加点击响应逻辑。
- 调用 `base.Start();` 是**必须**的，且**必须**在所有自定义参数设置完毕后调用，以正确初始化列表。

代码示例：`LoadingListAdapter.cs`

C#

```
//-----------------------------------------------------------------------------
// 业务逻辑部分 - 仅在初始创建时生成，请在此编写你的 Adapter 逻辑
//-----------------------------------------------------------------------------
using System.Collections.Generic;
using EUUI.Extension;
using UnityEngine;

```

```
namespace Game.UI
{
    ///
    /// LoadingListAdapter - List Adapter
    ///
    public class LoadingListAdapter : FrameworkListAdapter<LoadingListData, LoadingListVH>
    {
        protected override void Start()
        {
            // 在调用 base.Start() 之前配置参数
            // _Params.ItemPrefab = ... // 可重新设置列表项模板

            // 为列表项点击事件赋值
            OnItemClick = HandleItemClick;

            // 必须调用基类Start以完成初始化
            base.Start();
        }

        // 自定义的点击事件处理方法
        private void HandleItemClick(int index, LoadingListData data)
        {
            // Adapter 内处理点击逻辑，例如管理选中状态、数据变更等
            Debug.Log($"点击了第{index}项: {data.Title}");
        }
    }
}
```

美术处理UI资源

UI页面处理

创建UI页面之后，只需要把控件摆好，若是明确那些控件需要代码交互，则进行绑定操作。导出以及其他prefab处理程序会自行处理。即美术只需要处理好CreateUIScene的UIScene资源

列表处理：

只需要提供一个列表项的prefab在场景上即可（默认的控件接口），程序使用的会自己生成Adapter和VH

XXListItem(empty)。即美术只需要处理好列表元素项

- 控件 1
- 控件 2